

C# BACKEND TRACK | OOP PHASE

Independent Task

University System ERD & Models

Design the ERD. Build the models. Apply the relationships.

Codeline Bootcamp · Backend Phase · OOP Session 5 Task

01 DOMAIN OVERVIEW

You are given the requirements of a **University Management System**. A university has students enrolled in a study program, and instructors who teach courses. Each course belongs to a department, and a department has one instructor as its head. Students enroll in courses each semester, and for every enrollment a final grade is eventually recorded. An instructor teaches one or more courses, and a course can be taught by one instructor at a time. Below you will find the five main entities required for this system, each with its full list of attributes, data types, and constraints. Use this specification to complete the three parts of this task.

You must deliver three things:

1. A complete ERD diagram (entities, attributes, relationships, cardinality, participation)
2. A new C# project with a Models/ folder containing one class per entity, fully annotated
3. The relationships between classes implemented using navigation properties as taught in the session

02 ENTITIES, ATTRIBUTES & CONSTRAINTS

These five entities and their attributes are the system requirements. Use them as the source of truth for both your ERD and your C# models. The relationships between these entities are explained separately in Section 3 — you decide how to express them visually in the ERD and in code.

Student		
Attribute	Data Type	Constraints
studentId	int	Primary Key, auto-generated, not null
fullName	string	Required, max length 100
email	string	Required, unique, max length 150
phoneNumber	string	Optional, max length 20
dateOfBirth	DateTime	Required
enrollmentYear	int	Required, range 2000-2030
gpa	decimal	Default 0.0, range 0.0-4.0

Instructor		
Attribute	Data Type	Constraints
instructorId	int	Primary Key, auto-generated, not null
fullName	string	Required, max length 100
email	string	Required, unique, max length 150
officeNumber	string	Optional, max length 20
hireDate	DateTime	Required
salary	decimal	Required, must be greater than 0
academicTitle	string	Required, max length 50 (e.g. Professor, Lecturer)

Department		
Attribute	Data Type	Constraints
departmentId	int	Primary Key, auto-generated, not null
departmentName	string	Required, unique, max length 100
building	string	Optional, max length 50
budget	decimal	Required, must be greater than or equal to 0
headInstructorId	int	Foreign Key to Instructor, nullable (a department may have no head yet)

Course		
Attribute	Data Type	Constraints
courseId	int	Primary Key, auto-generated, not null
courseCode	string	Required, unique, max length 10 (e.g. CS101)
courseTitle	string	Required, max length 150
creditHours	int	Required, range 1-6
departmentId	int	Foreign Key to Department, not null
instructorId	int	Foreign Key to Instructor, nullable (a course may be unassigned)
semesterOffered	string	Required, max length 20 (e.g. Fall 2026)

Enrollment		
Attribute	Data Type	Constraints
enrollmentId	int	Primary Key, auto-generated, not null
studentId	int	Foreign Key to Student, not null
courseId	int	Foreign Key to Course, not null
enrollmentDate	DateTime	Required
finalGrade	string	Optional, max length 2 (e.g. A, B+, null until graded)
status	string	Required, max length 20, default "In Progress"

03 RELATIONSHIPS BETWEEN ENTITIES

Below is a plain-English description of how the five entities relate to each other in real life. Your job in Part 1 is to translate each of these into proper ERD notation - draw the correct cardinality symbols (1:1, 1:M, M:N) and participation type (total/mandatory vs partial/optional) for every relationship.

Department - Instructor (Department Head)

Description	Each department has exactly one instructor assigned as its head. Not every instructor is a department head, and a department may temporarily have no head assigned. An instructor can be the head of at most one department.
Cardinality	1 : 1
Participation	Department side: partial (a department may exist without a head). Instructor side: partial (most instructors are not heads of any department).

Department - Course

Description	Each course belongs to exactly one department. A department offers many courses.
Cardinality	1 : M
Participation	Course side: total (a course cannot exist without belonging to a department). Department side: partial (a newly created department may not yet have any courses).

Instructor - Course (Teaches)

Description	Each course is taught by one instructor at a time. An instructor can teach multiple courses in the same semester or across different semesters.
Cardinality	1 : M
Participation	Course side: partial (a course may temporarily have no instructor assigned). Instructor side: partial (an instructor may exist in the system without currently teaching any course).

Student - Course (Enrollment)	
Description	A student can enroll in many courses, and a course can have many students enrolled in it. This relationship is resolved through the Enrollment entity, which also stores additional data about the relationship itself (enrollment date, grade, status).
Cardinality	M : N (resolved via Enrollment)
Participation	Student side: partial (a student may be newly registered with no enrollments yet). Course side: partial (a course may have no students enrolled yet).

Student - Enrollment / Course - Enrollment	
Description	Enrollment is the bridge entity that breaks the Student-Course many-to-many relationship into two one-to-many relationships. Each enrollment record belongs to exactly one student and exactly one course.
Cardinality	1 : M (Student to Enrollment) and 1 : M (Course to Enrollment)
Participation	Enrollment side: total on both relationships (an enrollment record cannot exist without a specific student and a specific course). Student/Course side: partial (they can exist without any enrollment records).

04 PART 1 - DRAW THE ERD

Using the entities, attributes, and relationships above, produce a complete Entity-Relationship Diagram. Your ERD must include:

- All 5 entities, clearly labeled
- Every attribute listed inside (or attached to) its entity, with the primary key underlined or marked
- Foreign key attributes clearly indicated
- A connecting line for every relationship described in Section 3
- The correct cardinality notation on each end of every relationship line (1, M, or N)
- The correct participation type for every entity in every relationship (single line = partial/optional, double line = total/mandatory - or use min-max notation if you prefer Chen/Crow's Foot style)
- A legend explaining your notation choice if it is not the standard Crow's Foot notation

Tooling: Use any ERD tool you are comfortable with (draw.io, Lucidchart, dbdiagram.io, or even a clean hand-drawn diagram photographed/scanned). Export as PDF, PNG, or include the diagram tool link if shared (e.g. dbdiagram.io share link).

05 PART 2 - BUILD THE C# MODELS

Create a new C# console project. Inside it, create a **Models/** folder containing one class file per entity: **Student.cs**, **Instructor.cs**, **Department.cs**, **Course.cs**, **Enrollment.cs**.

Each class must include:

- Every attribute listed in Section 2, using the data type given (int, string, DateTime, decimal)
- The appropriate data annotation(s) for every constraint listed - [Key], [Required], [MaxLength], [Range], [Column], [ForeignKey], [DatabaseGenerated] - exactly as taught in the session
- A C# comment beside every single attribute stating its source tag (system generated / user input / default value / calculated / from list / foreign key)
- Correct namespace and using statements for System.ComponentModel.DataAnnotations and System.ComponentModel.DataAnnotations.Schema

No code is provided for this section. You already know the syntax for every annotation and the source-tag comment style from the previous sessions. Apply it here independently.

06 PART 3 - APPLY THE RELATIONSHIPS

Using the navigation property patterns taught in the session, connect the five models together so that every relationship from Section 3 is expressed in code - not just as a foreign key int, but with the matching navigation property on both sides.

For every relationship, make sure you have:

- The foreign key int property, annotated with [ForeignKey("NavPropertyName")]

- The single-object navigation property on the "many" side (e.g. Course to Department)
- The ICollection navigation property on the "one" side (e.g. Department to ICollection)
- For the Student-Course many-to-many relationship: do NOT create a direct collection between Student and Course - implement it correctly through the Enrollment entity exactly as explained for many-to-many relationships in the session material

Self-check before submitting: Pick any two related entities and ask yourself - "can I navigate from one object directly to the other without writing a FirstOrDefault search?" If the answer is yes for every relationship in Section 3, Part 3 is complete.

07 SUBMISSION

Deliverable	Format
ERD Diagram	PDF or PNG export, or shareable diagram tool link
C# Project	Full project folder (Models/ + .csproj), zipped or pushed to GitHub
Source tag comments	Inline // comments beside every attribute in every model file

Push the complete C# project (including the ERD file in a /docs or /erd folder at the root) to your GitHub repository under a new branch named **feature/university-erd-models**, and open a pull request linking this task for review.